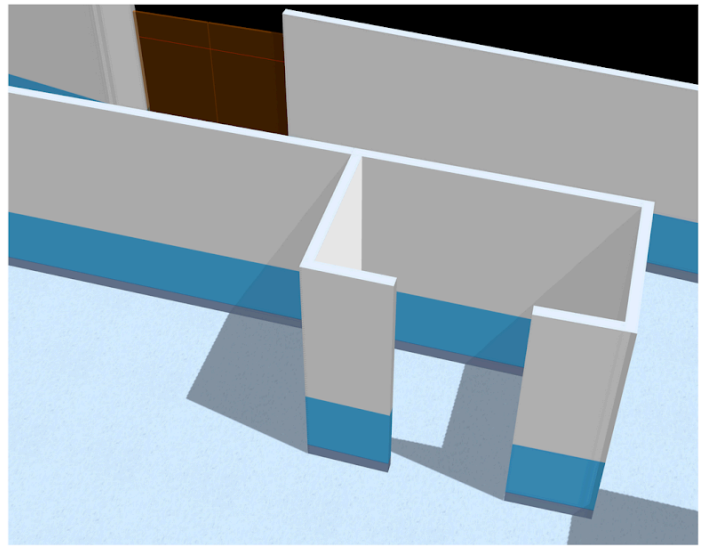
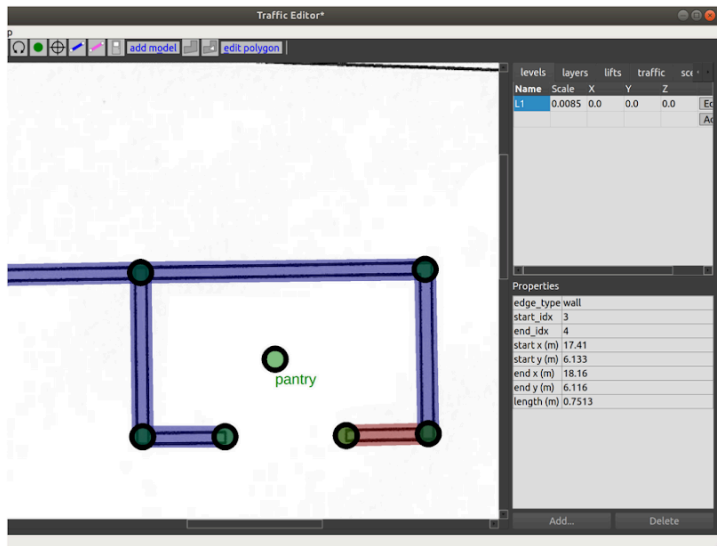
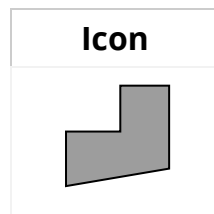




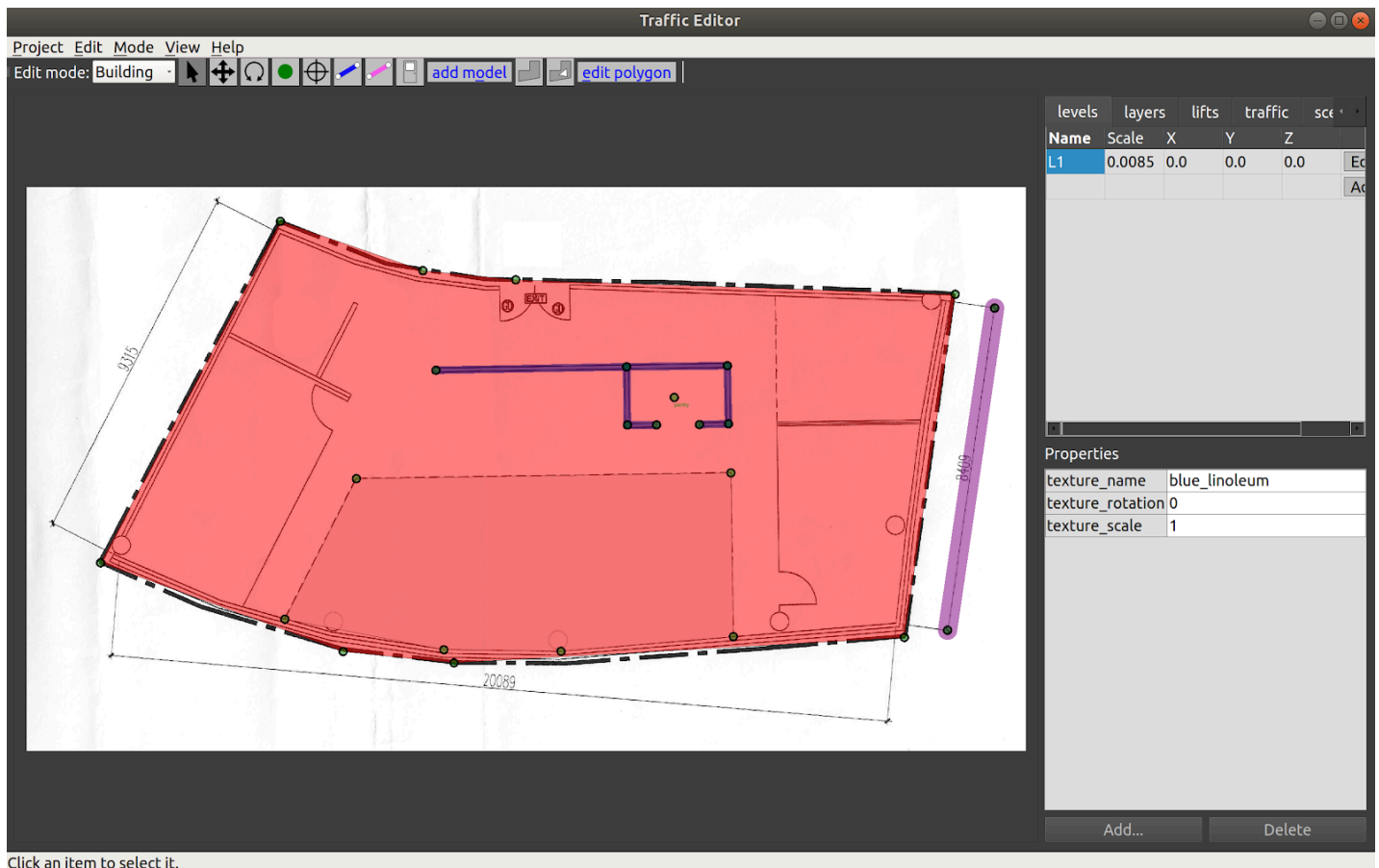
Walls are stored in the `tutorial.building.yaml` file as a list with indices of start and end vertices of the wall segment along with an empty parameter set.

```
walls:
- [3, 4, {}]
- [4, 5, {}]
- [5, 6, {}]
- [6, 7, {}]
- [6, 8, {}]
- [8, 9, {}]
```

Adding a floor



Flooring is essential for simulations as it provides a ground plane for the robots to travel over. Floors are annotated using the *Add floor polygon* tool from the **Main Toolbar** in *Building* edit mode. To define a floor, select consecutive vertices to create a polygon that accurately represents the flooring area as seen below. These vertices will need to be added manually prior to this step. Once created, save the project and reload. Selecting the defined floor highlights its texture attributes. Similarly, [default list of available textures](#) is available in the source code.

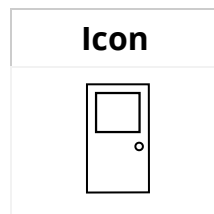


Certain scenarios may call for floors with cavities, for example, to represent elevator shafts. The *Add hole polygon* tool may be used for this purpose. Additionally, the shape of a drawn polygon (floor or hole) may be modified using the *Edit polygon* tool. Clicking on the tool after selecting an existing polygon enables the user to modify vertices of the polygon.

Each polygon is stored in the `tutorial.building.yaml` file in the format below:

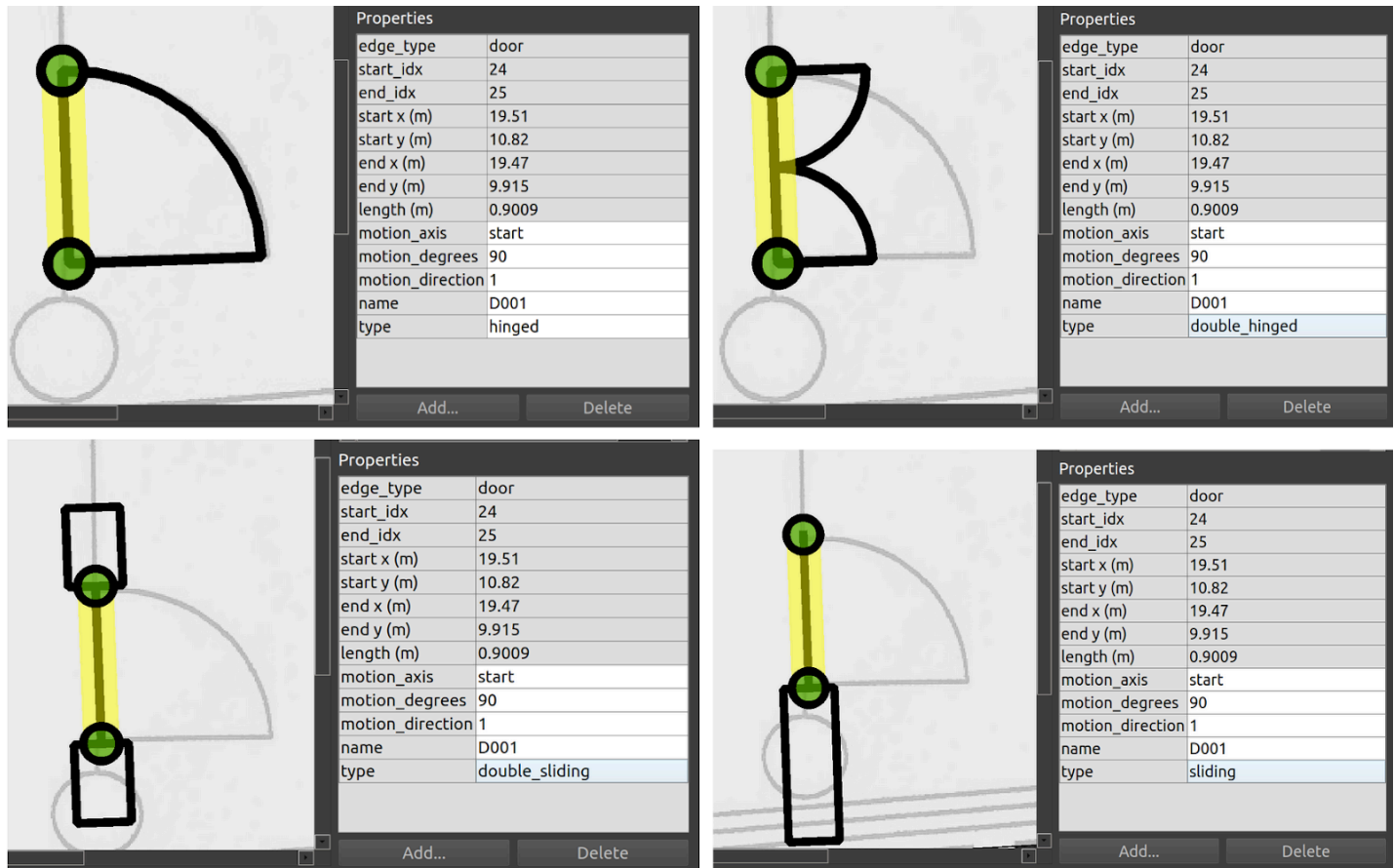
```
floors:
  - parameters: {texture_name: [1, blue_linoleum], texture_rotation: [3, 0],
    texture_scale: [3, 1]}
    vertices: [11, 10, 14, 15, 13, 12]
```

Adding a door



A door between two vertices can be added in *Building* edit mode by selecting the *Add door* tool from the **Main Toolbar**, and clicking on vertices representing the ends of the door. Selecting

an annotated door highlights its properties as seen in the figure below. Presently, four door types are supported: "hinged", "double_hinged", "sliding" and "double_sliding". The `motion_degrees` parameter specifies the range of motion in the case of hinged doors while the `motion_direction` dictates the direction of swing. In order for the door to work in simulation, a name must be given to the door.



Doors are stored in the `tutorial.building.yaml` file as a list with indices of start and end vertices along with the set of parameters that describes the door.

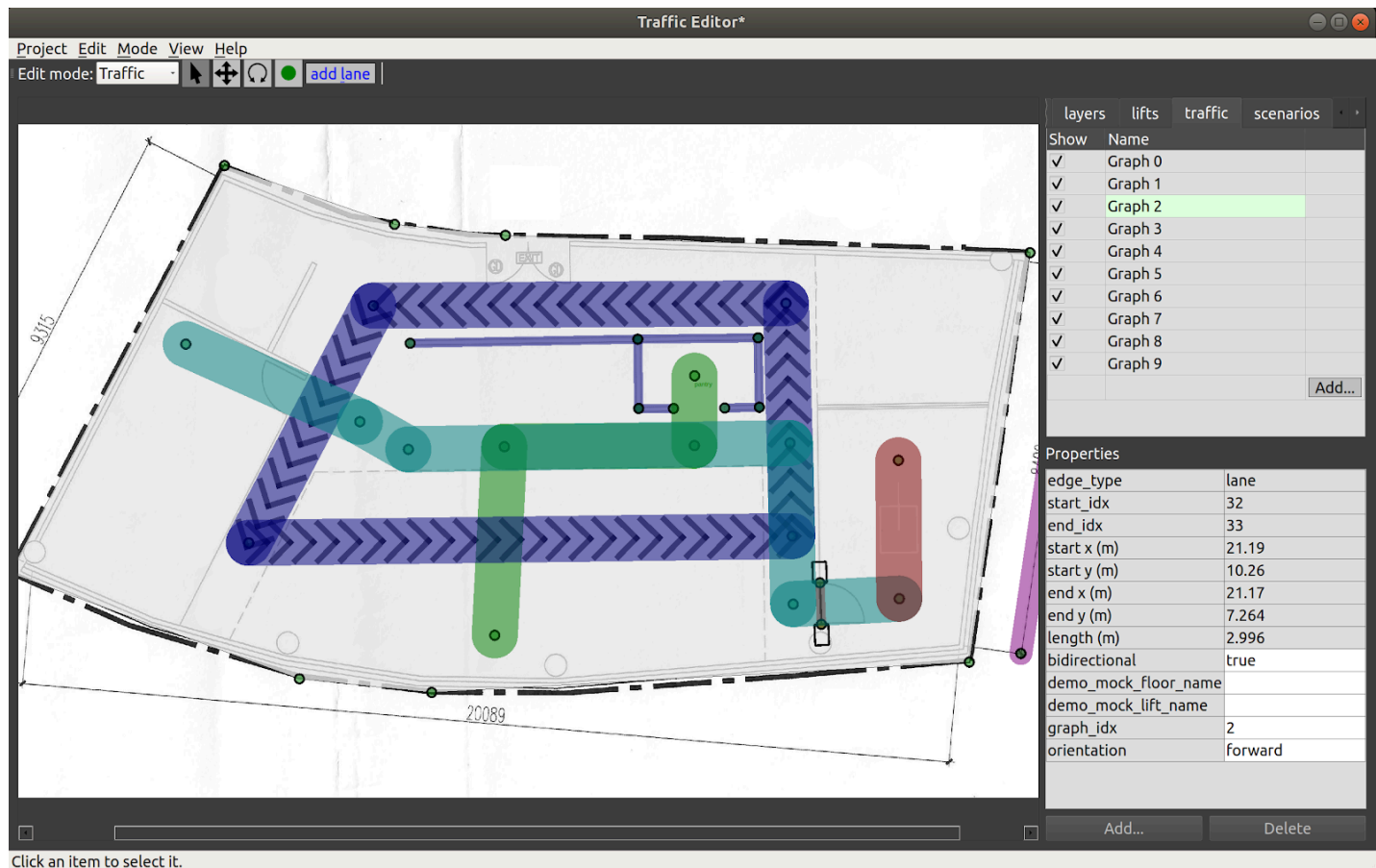
```
doors:
  - [24, 25, {motion_axis: [1, start], motion_degrees: [3, 90],
    motion_direction: [2, 1], name: [1, D001], type: [1, double_sliding]]]
```

Adding a traffic lane

One of the most important tools in the `traffic_editor` GUI is the *Add lane* tool. The allowable motions of each fleet operating in the facility is conveyed through its respective Graph which consists of waypoints and connecting lanes. In this approach, we assume that robots travel along effectively straight-line paths between waypoints. While this may be perceived as an oversimplification of paths taken by robots that are capable of autonomous navigation, in practice the assumption holds fairly well given that these robots mostly travel along corridors

or hallways and seldom in unconstrained open spaces. For example, even in theoretically unconstrained spaces like building lobbies or shopping-mall atriums, it is likely that the site operator would prefer for the robots to operate in a "traffic lane" on the edge of the space, in order to not impede typical human traffic flows.

The `traffic` tab in the `Sidebar` has a default of nine Graphs for nine different fleets. To annotate lanes for a graph, say Graph 0, select the Graph from the `traffic` tab and click the `Add lane` tool. Lanes for this graph can be drawn by clicking vertices to be connected. If a vertex is not present, it will automatically be added. Properties may be assigned to each vertex as described in the preceding section. To issue tasks to waypoints that require the robot to terminate at any waypoint, a name must be assigned to the waypoint.



Each Graph has a unique color for its lanes, and their visibility may be toggled using the checkbox in the `traffic` tab. A lane that is defined between two waypoints may be configured with these additional properties:

- **bidirectional:** if `true`, the `rmf_fleet_adapter` will plan routes for its robot assuming the lanes can be traversed in both directions. Lanes that are not bidirectional have arrows indicating their directionality (indigo lanes in figure above). A handy shortcut is that when a lane segment is selected, you can press the `b` key to toggle between unidirectional and bidirectional motion along that lane.
- **graph_idx:** the Graph number a lane corresponds to

- **orientation:** constrain the lane to make the robot travel in `forward` or `backward` orientation. This can be useful for the final lane segment approaching a docking point or charger, for example.

While lifts that move between levels are now supported in the `traffic_editor`, the **`demo_mock_floor_name`** and **`demo_mock_lift_name`** properties were originally engineered to showcase shared lift access in a single floor demonstration environment with a "mock" lift that receives lift commands and transmits lift states but does not actually move between any different floors in a building. However, as there may be interest in such functionality for testing single-floor hardware setups that seek to emulate multi-floor scenarios, these properties were retained.

- **`demo_mock_floor_name`:** name of the floor that the robot is on while traversing the lane
- **`demo_mock_lift_name`:** name of the lift that is being entered or exited while the robot traverses the lane

To further explain these properties, consider this representation of a navigation graph where numbers are waypoints and letters are lanes:

```
1 <---a---> 2 <---b---> 3
```

Waypoint 1 is on floor L1

Waypoint 2 is inside the "lift" named LIFT001

Waypoint 3 is on floor L3

The properties of edge "a" are:

`bidirectional: true`

`demo_mock_floor_name: L1`

`demo_mock_lift_name: LIFT001`

The properties of edge "b" are:

`bidirectional: true`

`demo_mock_floor_name: L3`

`demo_mock_lift_name: LIFT001`

If the robot is to travel from waypoint 1 to waypoint 3, the `rmf_fleet_adapter` will request for the "mock lift" to arrive at L1 when the robot approaches waypoint 1. With confirmation of the "lift" at L1 and its doors in "open" state, the robot will be instructed to move into the "lift" to waypoint 2. Once the "lift" indicates that it has reached L3, the robot will exit along lane b toward waypoint 3.

Note: when annotating graphs, it is highly recommended to follow an ascending sequence of graph indices without skipping intermediate numbers. Drawn lanes can only be interacted with if their associated Graph is first selected in the `traffic` tab.

The annotated Graphs are eventually exported as `navigation graphs` using the `building_map_generator` which are then used by respective `rmf_fleet_adapters` for path planning.

Lanes are stored in the following format in `tutorial.building.yaml`. The data structure is a list with the first two elements representing the indices of the two vertices of the lane and a set of parameters with configured properties.

```
lanes:
  - [32, 33, {bidirectional: [4, true], demo_mock_floor_name: [1, ""],
demo_mock_lift_name: [1, ""], graph_idx: [2, 2], orientation: [1, forward]}]
```

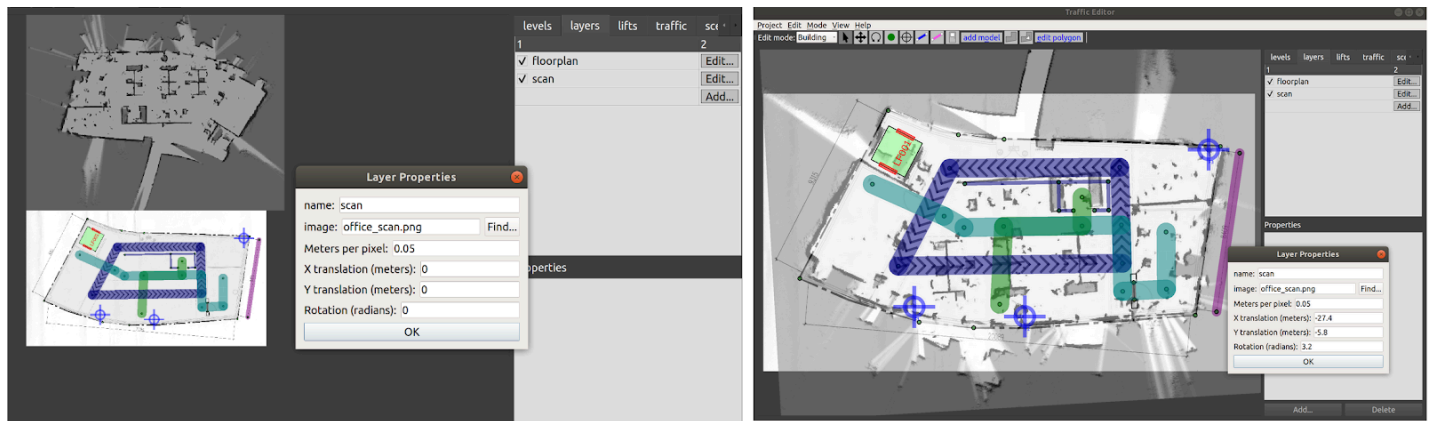
Deriving coordinate-space transforms

Coordinate spaces are confusing! For historical reasons, the GUI internally creates traffic maps by annotating images, so the "raw" annotations are actually encoded in pixel coordinates of the "base" floorplan image, in pixel coordinates, with +X=right and +Y=down, with the origin in the upper-left of the base floorplan image. However, during the building map generation step, the vertical axis is flipped to end up in a Cartesian plane, so the vast majority of RMF (that is, everything downstream of traffic-editor and the building map generators) uses a "normal" Cartesian coordinate system. (As an aside -- the next version of traffic-editor is intended to be more flexible in this respect, and will default to a normal Cartesian coordinate system (not an image-based coordinate system), or even global coordinates (lat/lon). Although preliminary work is underway, there is not a hard schedule for this next-gen editor at time of writing, so the rest of this chapter will describe the existing `traffic-editor`.)

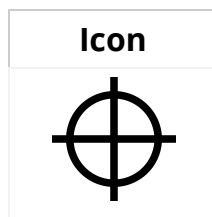
Although `traffic-editor` currently uses the upper-left corner of the base floorplan image as the reference frame, maps generated by robots likely will have their origin elsewhere, and will likely be oriented and scaled differently. It is critical to derive the correct transform between coordinate frames in `traffic_editor` maps and robot maps as `rmf_fleet_adapters` expect all robots to publish their locations in the RMF coordinate system while the `rmf_fleet_adapters` also issue path requests in the same frame.

To derive such transforms, the `traffic_editor` GUI allows users to overlay robot maps on a floor plan and apply scale, translation and rotation transformations such that the two maps align correctly. The user can then apply the same transformations to convert between robot map and RMF coordinates when programming interfaces for their robot.

The robot map can be imported by clicking the `Add` button from the `Layers` tab in the `Sidebar`. A dialog box will then prompt the user to upload the robot map image. The same box contains fields for setting the scale for the image along with applying translations and rotation. Through visual feedback, the user can determine appropriate values for these fields. As seen in the image below, importing the robot-generated map into the GUI has it located and oriented differently than the floor plan. With the right transformation values, the two maps can be made to overlap.

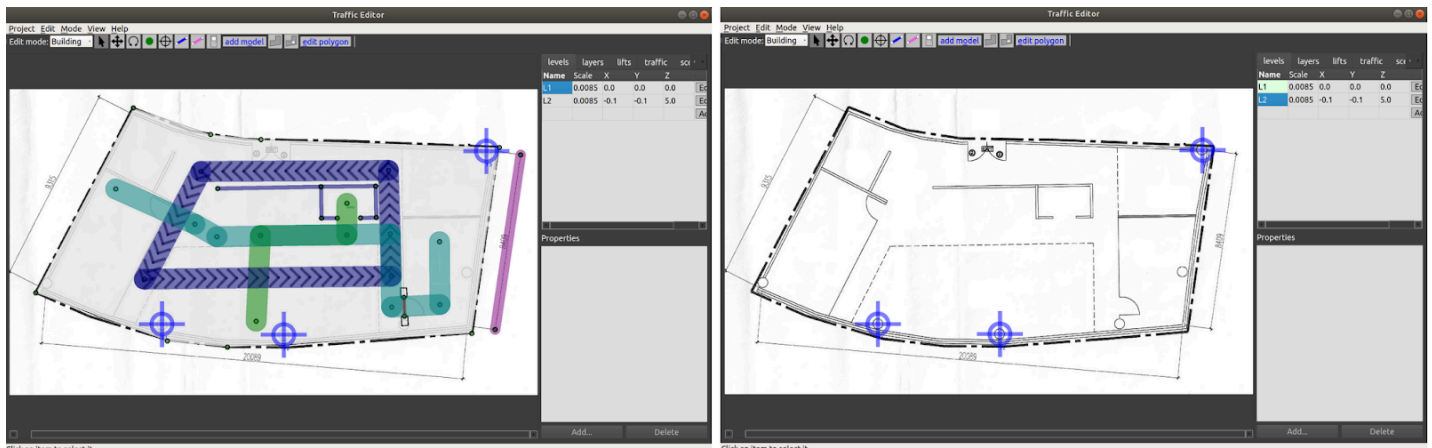


Adding fiducials



For maps with multiple levels, fiducials provide a means to scale and align different levels with respect to a reference level. This is crucial for ensuring dimensional accuracy of annotations across different levels and aligning the same for simulation. Fiducials are reference markers placed at locations which are expected to be vertically aligned between two or more levels. For example, structural columns may run through multiple floors and their locations are often indicated on floor plans. With two or more pairs of corresponding markers between a level and a reference level, a geometric transformation (translation, rotation and scale) may be derived between the two levels. This transformation can then be applied to all the vertices and models in the newly defined level.

To begin, add two or more non-collinear fiducials to the reference level with unique `name` attributes using the *Add fiducial* tool (left image in figure below). In the newly created level, add the same number of fiducials at locations that are expected to be vertically aligned with matching names as the reference level (right image in figure below). Saving and reloading the project computes the transformation between the levels which is evident from the Scale and X-Y offsets for the new level as seen in the `levels` tab. This level is now ready to be annotated.



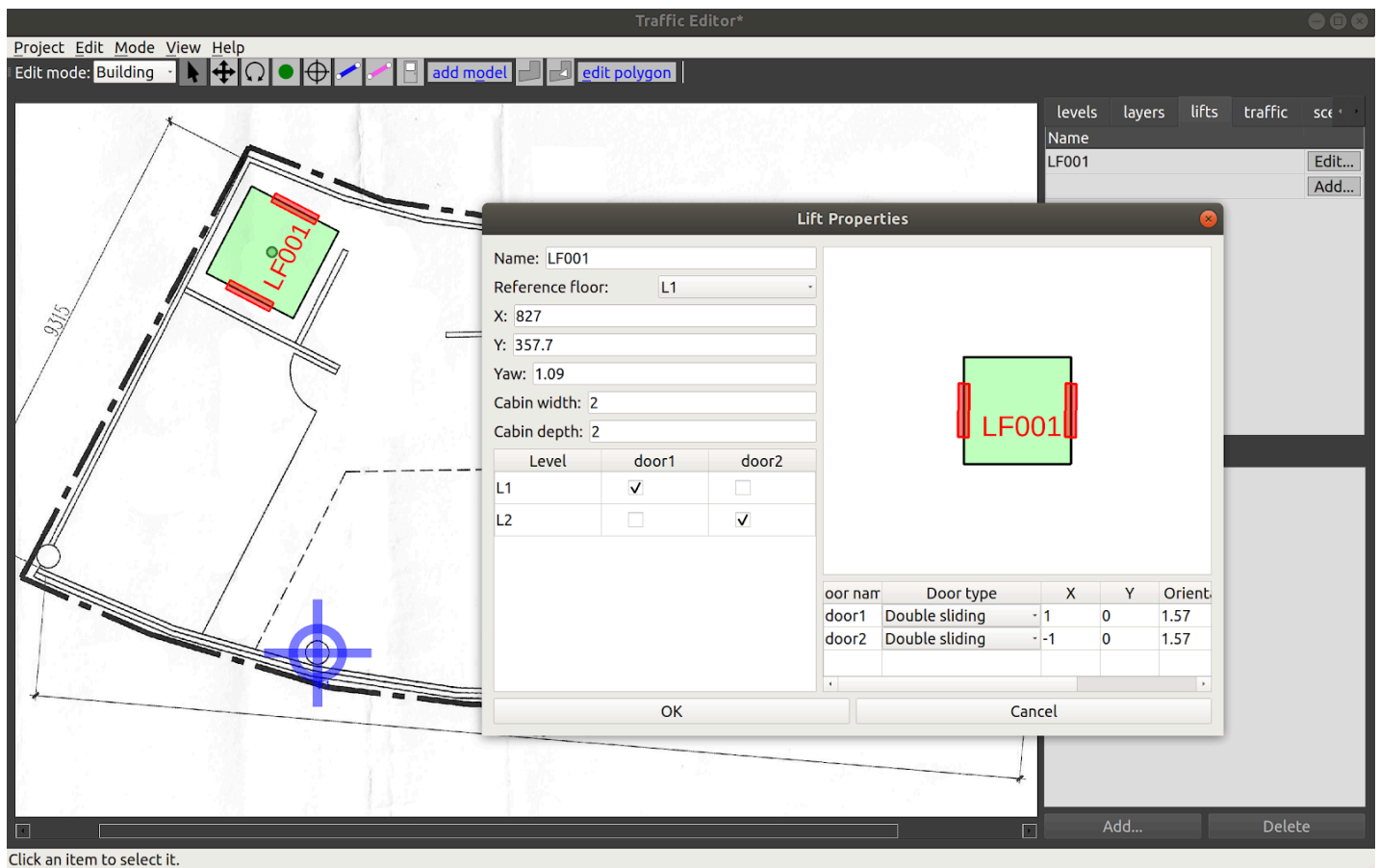
For each level, fiducials are stored in a list of their X & Y coordinates along with their name.

fiducials:

- [936.809, 1323.141, F1]
- [1622.999, 1379.32, F2]
- [2762.637, 346.69, F3]

Adding a lift

Lifts are integral resources that are shared between humans and robot fleets in multi-level facilities. To add a lift to a building, click the **Add** button in the **lifts** tab in the **sidebar**. A dialog box with various configurable properties will load. It is essential to specify the Name, Reference level and the X&Y coordinates (pixel units) of its cabin center. A yaw (radians) may further be added to orient the lift as desired. The width and depth of the cabin (meters) can also be customized. Lifts can be designed to have multiple cabin doors which may open at more than one level. To add a cabin door, click the **Add** button in the box below the cabin image. Each cabin door requires a name along with positional and orientational information. Here, the X&Y coordinates are relative to the cabin center.



The configured lift is stored in the `tutorial.building.yaml` file as described below:

```
lifts:
  LF001:
    depth: 2
    doors:
      door1:
        door_type: 2
        motion_axis_orientation: 1.57
        width: 1
        x: 1
        y: 0
      door2:
        door_type: 2
        motion_axis_orientation: 1.57
        width: 1
        x: -1
        y: 0
    level_doors:
      L1: [door1]
      L2: [door2]
    reference_floor_name: L1
    width: 2
    x: 827
    y: 357.7
    yaw: 1.09
```

After adding the lift, we would also wish to let our robots to transverse through the lift. To achieve that, the user needs to create vertices/waypoints which are located within the lift cabin on each floor. Once done, connect the waypoint within the lift cabin to other vertices via **add_lane**.

Adding environment assets

Levels may be annotated with thumbnails of models available for simulation using the **Add model** tool in **Building** edit mode. Selecting this tool opens a dialog box with a list of model names and matching thumbnails which can be imported to the map. Once on the map, their positions and orientations can be adjusted using the *Move* and *Rotate* tools. Sample models are provided [here](#)

The [thumbnail_generator documentation](#) contains instructions on expanding the list of thumbnails for other models.

Note: If no models are shown on the **add models** window, Go to "Edit -> Preference", then indicate the thumbnail path. (e.g.

```
$HOME/rmf_ws/src/rmf/rmf_traffic_editor/rmf_traffic_editor_assets/assets/thumbnails )
```
